

"""

ربات جامع تحلیل ارز دیجیتال + رصد نهنگ + اطلاع رسانی تلگرام

=====

:این نسخه یه ربات کامل و همیشه-روشن هست که

1. حجم, (RSI, MACD) با فرستادن دستور توی تلگرام، تحلیل تکنیکال کامل، حمایت/مقاومت, سیگنال خرید/فروش) رو همراه با نمودار برات می‌فرسته
2. معاملات درشت (نهنگ) روی بایننس رو لحظه‌ای رصد می‌کنه و به محض دیدن یه معامله بزرگ، فوری بهت پیام تلگرام میده
3. هر مدت مشخص (پیش‌فرض هر ۶۰ دقیقه) خودش خودکار نمادهای تحت رصد رو چک می‌کنه و اگه سیگنال قوی خرید/فروش دید، بهت خبر میده

نکته مهم: این ربات باید ۲۴ ساعته روشن بمونه تا کارش رو درست انجام بده !  
رایگان مناسب نیست (چون با بستن صفحه متوقف میشه) Replit روی گوشی یا Always-On با پلن Railway/Render ارزون (ماهی ۵-۱۰ دلار) یا VPS پیشنهاد: یه

---

راه اندازی

---

۱) نصب پیش‌نیازها:

```
pip install ccxt pandas numpy mplfinance requests websocket-client
```

۲) ساخت بات تلگرام:

- @BotFather توی تلگرام برو سراغ
- و اسم دلخواه بده /newbot بزن
- AAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx:یه توکن بهت میده شبیه این: 123456789
- پایین همین فایل TELEGRAM\_BOT\_TOKEN اون توکن رو بذار توی متغیر

۳) اجرا:

```
python pro_crypto_bot.py
```

۴) /start توی تلگرام برو سراغ باتی که ساختی و بزن

(تو رو یاد بگیره و ازین به بعد بتونه پیام بده chat\_id این کار باعث میشه ربات)

۵) دستورات قابل استفاده توی تلگرام:

|                   |                                                    |
|-------------------|----------------------------------------------------|
| /start            | فعالسازی و دیدن راهنما                             |
| /analyze BTC/USDT | تحلیل کامل + نمودار                                |
| /watch SOLUSDT    | اضافه کردن نماد به رصد نهنگ (بدون اسلش، حروف بزرگ) |
| /unwatch SOLUSDT  | حذف نماد از رصد نهنگ                               |
| /watchlist        | نمایش نمادهای تحت رصد                              |
| /whales_on        | روشن کردن هشدار نهنگ                               |
| /whales_off       | خاموش کردن هشدار نهنگ                              |
| /help             | راهنما                                             |

"""

```

import os
import sys
import json
import time
import threading
from datetime import datetime

import requests
import ccxt
import numpy as np
import pandas as pd
import mplfinance as mpf
import websocket # از پکیج websocket-client

# =====
# تنظیمات - این بخش رو خودت پر کن
# =====

TELEGRAM_BOT_TOKEN = "8891450951:AAEqBKslx3mzxzeZUt-rW7Tv8PHXYWX8zm8"

WATCHED_SYMBOLS_DEFAULT = ["BTCUSD", "ETHUSD"] # نمادهای پیشفرض برای رصد نهنگ
WHALE_USD_THRESHOLD = 100_000 # برای شمردنش به عنوان "نهنگ"
AUTO_ANALYSIS_INTERVAL_MIN = 60 # ر خودکار سیگنالها رو چک کنه

STATE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "bot_state.json")
OUTDIR = os.path.dirname(os.path.abspath(__file__))

TELEGRAM_API = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}"
ws_thread_ref = {"current": None}

# =====
# روی دیسک تا با ریاستارت از بین نره (لیست رصد, و غیره, chat_id, ذخیره وضعیت)
# =====

def load_state():
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            pass
    return {"chat_id": None, "watch": list(WATCHED_SYMBOLS_DEFAULT), "whales_enab

```

```

def save_state(s):
    with open(STATE_FILE, "w", encoding="utf-8") as f:
        json.dump(s, f, ensure_ascii=False, indent=2)

state = load_state()
state_lock = threading.Lock()

# =====
# ارتباط با تلگرام
# =====

def tg_send_message(text, chat_id=None):
    chat_id = chat_id or state.get("chat_id")
    if not chat_id:
        print("[تلگرام] هنوز chat_id باید /start بزنیم.")
        return
    try:
        requests.post(f"{TELEGRAM_API}/sendMessage", data={
            "chat_id": chat_id, "text": text, "parse_mode": "HTML"
        }, timeout=15)
    except Exception as e:
        print(f"[تلگرام] خطا در ارسال پیام: {e}")

def tg_send_photo(path, caption="", chat_id=None):
    chat_id = chat_id or state.get("chat_id")
    if not chat_id or not os.path.exists(path):
        return
    try:
        with open(path, "rb") as f:
            requests.post(f"{TELEGRAM_API}/sendPhoto", data={
                "chat_id": chat_id, "caption": caption
            }, files={"photo": f}, timeout=30)
    except Exception as e:
        print(f"[تلگرام] خطا در ارسال عکس: {e}")

# =====
# داده و اندیکاتورها (تحلیل تکنیکال)
# =====

def fetch_ohlc(exchange, symbol, timeframe, limit=300):
    data = exchange.fetch_ohlc(symbol, timeframe=timeframe, limit=limit)
    df = pd.DataFrame(data, columns=["timestamp", "open", "high", "low", "close",
    df["timestamp"] = pd.to_datetime(df["timestamp"], unit="ms")

```

```

return df

def resample_5d(df_daily):
    df = df_daily.set_index("timestamp")
    df5d = df.resample("5D").agg({
        "open": "first", "high": "max", "low": "min",
        "close": "last", "volume": "sum",
    }).dropna()
    return df5d.reset_index()

def compute_indicators(df):
    d = df.copy()
    d["sma20"] = d["close"].rolling(20).mean()
    d["sma50"] = d["close"].rolling(50).mean()

    ema12 = d["close"].ewm(span=12, adjust=False).mean()
    ema26 = d["close"].ewm(span=26, adjust=False).mean()
    d["macd"] = ema12 - ema26
    d["macd_signal"] = d["macd"].ewm(span=9, adjust=False).mean()

    delta = d["close"].diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.rolling(14).mean()
    avg_loss = loss.rolling(14).mean()
    rs = avg_gain / avg_loss.replace(0, np.nan)
    d["rsi"] = (100 - (100 / (1 + rs))).fillna(50)

    d["bb_mid"] = d["close"].rolling(20).mean()
    bb_std = d["close"].rolling(20).std()
    d["bb_upper"] = d["bb_mid"] + 2 * bb_std
    d["bb_lower"] = d["bb_mid"] - 2 * bb_std

    d["vol_avg20"] = d["volume"].rolling(20).mean()
    return d

def find_pivots(df, window=5):
    highs, lows = [], []
    for i in range(window, len(df) - window):
        seg_high = df["high"].iloc[i - window:i + window + 1]
        seg_low = df["low"].iloc[i - window:i + window + 1]
        if df["high"].iloc[i] == seg_high.max():
            highs.append(df["high"].iloc[i])
        if df["low"].iloc[i] == seg_low.min():

```

```

        lows.append(df["low"].iloc[i])
    return sorted(set(highs), reverse=True), sorted(set(lows))

def score_row(row, prev):
    bull, bear = 0, 0
    rb, rs = [], []
    if row["rsi"] < 35:
        bull += 1; rb.append(f"RSI اشباع فروش ({row['rsi']:.0f})")
    if row["rsi"] > 65:
        bear += 1; rs.append(f"RSI اشباع خرید ({row['rsi']:.0f})")
    if row["macd"] > row["macd_signal"] and prev["macd"] <= prev["macd_signal"]:
        bull += 1; rb.append("MACD کراس صعودی")
    if row["macd"] < row["macd_signal"] and prev["macd"] >= prev["macd_signal"]:
        bear += 1; rs.append("MACD کراس نزولی")
    if row["close"] <= row["bb_lower"]:
        bull += 1; rb.append("قیمت روی باند پایین بولینگر")
    if row["close"] >= row["bb_upper"]:
        bear += 1; rs.append("قیمت روی باند بالای بولینگر")
    if row["sma20"] > row["sma50"] and prev["sma20"] <= prev["sma50"]:
        bull += 1; rb.append("کراس صعودی میانگین متحرک")
    if row["sma20"] < row["sma50"] and prev["sma20"] >= prev["sma50"]:
        bear += 1; rs.append("کراس نزولی میانگین متحرک")
    if pd.notna(row["vol_avg20"]) and row["volume"] > row["vol_avg20"] * 1.5:
        if row["close"] > prev["close"]:
            bull += 1; rb.append("افزایش حجم همراه رشد قیمت")
        elif row["close"] < prev["close"]:
            bear += 1; rs.append("افزایش حجم همراه افت قیمت")
    return bull, bear, rb, rs

def generate_signals(df, threshold=2):
    d = df.copy()
    d["buy_signal"] = False
    d["sell_signal"] = False
    for i in range(1, len(d)):
        bull, bear, _, _ = score_row(d.iloc[i], d.iloc[i - 1])
        if bull >= threshold and bull > bear:
            d.at[d.index[i], "buy_signal"] = True
        if bear >= threshold and bear > bull:
            d.at[d.index[i], "sell_signal"] = True
    return d

def plot_chart(df, symbol, timeframe_name):
    d = df.set_index("timestamp").rename(columns={
        "open": "Open", "high": "High", "low": "Low", "close": "Close", "volume":

```

```

    })
    buy_marks = d["Low"].where(d["buy_signal"]) * 0.99
    sell_marks = d["High"].where(d["sell_signal"]) * 1.01
    apds = []
    if buy_marks.notna().any():
        apds.append(mpf.make_addplot(buy_marks, type="scatter", markersize=90, ma
    if sell_marks.notna().any():
        apds.append(mpf.make_addplot(sell_marks, type="scatter", markersize=90, m
    safe = symbol.replace("/", "_")
    filename = os.path.join(OUTDIR, f"{safe}_{timeframe_name}.png")
    mpf.plot(
        d[["Open", "High", "Low", "Close", "Volume"]], type="candle", volume=True
        addplot=apds if apds else None, style="yahoo",
        title=f"{symbol} - {timeframe_name}", savefig=dict(fname=filename, dpi=15
    )
    return filename

def volume_report_text(df_ld):
    lines = [" <b>تحلیل حجم معاملات</b>"]
    if len(df_ld) < 2:
        lines.append("داده کافی نیست.")
        return "\n".join(lines)
    today = df_ld["volume"].iloc[-1]
    week_avg = df_ld["volume"].tail(7).mean()
    month_avg = df_ld["volume"].tail(30).mean()
    lines.append(f"حجم امروز: {today:,.2f}")
    lines.append(f"میانگین هفته: {week_avg:,.2f}")
    lines.append(f"میانگین ماه: {month_avg:,.2f}")

    def cmp_txt(val, ref):
        diff = (val - ref) / ref * 100 if ref else 0
        s = "پایین‌تر" if diff > 5 else "مشابه" if diff < -5 else "پایین‌تر"
        return f"{s} ({diff:+.1f}%)"

    lines.append(f"نسبت به میانگین هفته: {cmp_txt(today, week_avg)}")
    lines.append(f"نسبت به میانگین ماه: {cmp_txt(today, month_avg)}")
    if today > month_avg * 1.5:
        lines.append("حجم غیرعادی بالا - احتمال شروع یک حرکت قوی ▲")
    elif today < month_avg * 0.5:
        lines.append("حجم پایین‌تر از حد معمول - بازار کم‌نوسان")
    return "\n".join(lines)

def btc_correlation_text(df_coin, df_btc):
    n = min(len(df_coin), len(df_btc))
    if n < 15:

```

```

        return ""
    coin_ret = df_coin["close"].tail(n).pct_change().dropna()
    btc_ret = df_btc["close"].tail(n).pct_change().dropna()
    m = min(len(coin_ret), len(btc_ret))
    if m < 10:
        return ""
    coin_ret = coin_ret.tail(m).reset_index(drop=True)
    btc_ret = btc_ret.tail(m).reset_index(drop=True)
    corr = coin_ret.corr(btc_ret)
    if pd.isna(corr):
        return ""
    btc_var = btc_ret.var()
    beta = (coin_ret.cov(btc_ret) / btc_var) if btc_var else float("nan")
    btc_change = (df_btc["close"].iloc[-1] - df_btc["close"].iloc[-2]) / df_btc["close"].iloc[-1]
    desc = ("خیلی بالا" if corr >= 0.75 else "نسبتا بالا" if corr >= 0.5
            else "متوسط" if corr >= 0.2 else "پایین")
    lines = [f"همبستگی با بیت کوین: {corr:.2f} ({desc})"]
    if not pd.isna(beta):
        lines.append(f"بتا: {beta:.2f} - بیت کوین اخیرا {btc_change:+.2f}% → مینی")
    return "\n".join(lines)

def analyze_one_timeframe(df, name, symbol, df_btc=None, is_btc=False, make_chart=True):
    """وضعیت کلی، متن گزارش، مسیر فایل نمودار یا: برمی‌گردونه"""
    if len(df) < 55:
        return f"=== {name} ===\nکافی نیست", None, "neutral"

    d = compute_indicators(df)
    d = generate_signals(d)
    last, prev = d.iloc[-1], d.iloc[-2]
    change_pct = (last["close"] - prev["close"]) / prev["close"] * 100
    recent = d.tail(60)
    resistance = recent["high"].max()
    support = recent["low"].min()
    ph, pl = find_pivots(d.tail(100), window=3)

    bull, bear, rb, rs = score_row(last, prev)
    if bull >= 2 and bull > bear:
        overall, reasons, status = "🟢 سیگنال خرید", rb, "buy"
    elif bear >= 2 and bear > bull:
        overall, reasons, status = "🔴 سیگنال فروش", rs, "sell"
    else:
        overall, reasons, status = "🟡 [اندیکاتورها هم‌جهت نیستن]", "خنثی", "neutral"

    lines = [f"=== تایم‌فریم {name} ===",
             f"قیمت: {last['close']:.6f} ({change_pct:+.2f}%)",
             f"RSI: {last['rsi']:.1f} | MACD: {last['macd']:.6f}/{last['macd_sign']:.1f}"]

```

```

        f"SMA20/50: {last['sma20']:.6f} / {last['sma50']:.6f}",
        f"مقاومت نزدیک: {resistance:.6f} | حمایت نزدیک: {support:.6f}"]
if ph[:3]:
    lines.append("مقاومت‌های مهم: " + ", ".join(f"{v:.6f}" for v in ph[:3]))
if pl[:3]:
    lines.append("حمایت‌های مهم: " + ", ".join(f"{v:.6f}" for v in pl[:3]))
lines.append(f"\nوضعیت: {overall}")
lines += [f" - {r}" for r in reasons]

if df_btc is not None and not is_btc:
    bt = btc_correlation_text(df, df_btc)
    if bt:
        lines.append("\nنایر بیت کوین:")
        lines.append(bt)

chart_path = None
if make_chart:
    try:
        chart_path = plot_chart(d, symbol, name)
    except Exception as e:
        lines.append(f"خطا در رسم نمودار: {e}")

return "\n".join(lines), chart_path, status

def run_full_analysis_and_send(symbol, chat_id):
    """اجرای تحلیل کامل روی هر سه تایم‌فریم و ارسال به تلگرام"""
    exchange = ccxt.binance()
    is_btc = symbol.upper().startswith("BTC/")

    df_4h = fetch_ohlcv(exchange, symbol, "4h")
    df_1d = fetch_ohlcv(exchange, symbol, "1d")
    df_5d = resample_5d(df_1d)

    df_btc_4h = df_btc_1d = df_btc_5d = None
    if not is_btc:
        try:
            df_btc_4h = fetch_ohlcv(exchange, "BTC/USDT", "4h")
            df_btc_1d = fetch_ohlcv(exchange, "BTC/USDT", "1d")
            df_btc_5d = resample_5d(df_btc_1d)
        except Exception:
            pass

    tg_send_message(volume_report_text(df_1d), chat_id=chat_id)

    for df, name, dbtc in [(df_4h, "4ساعته", df_btc_4h), (df_1d, "روزانه", df_btc_1d), (df_5d, "5روزه", df_btc_5d)]:
        text, chart_path, _ = analyze_one_timeframe(df, name, symbol, df_btc=dbtc)

```

```

tg_send_message(text, chat_id=chat_id)
if chart_path:
    tg_send_photo(chart_path, caption=f"{symbol} - {name}", chat_id=chat_id)

tg_send_message("[یادآوری: این تحلیل تضمینی نیست، مدیریت ریسک رو فراموش نکن]")

# =====
# WebSocket رصد نهنگ - معاملات درشت روی بایننس، لحظه ای از طریق
# =====

def build_stream_url(symbols):
    streams = "/".join(f"{s.lower()}@aggTrade" for s in symbols)
    return f"wss://stream.binance.com:9443/stream?streams={streams}"

def on_ws_message(ws, message):
    try:
        payload = json.loads(message)
        data = payload.get("data", {})
        price = float(data.get("p", 0))
        qty = float(data.get("q", 0))
        is_seller_taker = data.get("m", False) # True یعنی فروشنده taker (فروش)
        symbol = data.get("s", "")
        value_usd = price * qty
        if value_usd >= WHALE_USD_THRESHOLD and state.get("whales_enabled", True):
            emoji = "🔴" if is_seller_taker else "🟢"
            direction = "فروش بزرگ" if is_seller_taker else "خرید بزرگ"
            text = (f"{emoji} <b>معامله نهنگ</b> - {symbol}</b>\n"
                    f"نوع: {direction}\n"
                    f"قیمت: {price:,.4f}\n"
                    f"مقدار: {qty:,.4f}\n"
                    f"ارزش: ${value_usd:,.0f}\n"
                    f"زمان: {datetime.now().strftime('%H:%M:%S')}")
            print(text.replace("<b>", "").replace("</b>", ""))
            tg_send_message(text)
    except Exception as e:
        print(f"[نهنگ] خطا در پردازش پیام: {e}")

def on_ws_error(ws, error):
    print(f"[نهنگ] خطای اتصال: {error}")

def on_ws_close(ws, code, msg):
    print("[نهنگ] اتصال قطع شد.")

```

```

def on_ws_open(ws):
    print(f"[نهنگ] رصد فعال برای {state.get('watch')}")

def whale_watcher_loop():
    while True:
        symbols = state.get("watch") or WATCHED_SYMBOLS_DEFAULT
        url = build_stream_url(symbols)
        ws = websocket.WebSocketApp(
            url, on_message=on_ws_message, on_error=on_ws_error,
            on_close=on_ws_close, on_open=on_ws_open,
        )
        ws_thread_ref["current"] = ws
        try:
            ws.run_forever(ping_interval=30, ping_timeout=10)
        except Exception as e:
            print(f"[نهنگ] خطای غیرمنتظره: {e}")
            time.sleep(5) # قبل از تلاش مجدد کمی صبر کن

# =====
# بررسی خودکار دوره ای نمادهای تحت رصد و اطلاع رسانی سیگنال قوی
# =====

last_alert_status = {} # symbol -> دادیم که اطلاع آخرین وضعیتی که اطلاع دادیم

def auto_analysis_loop():
    while True:
        time.sleep(AUTO_ANALYSIS_INTERVAL_MIN * 60)
        symbols = state.get("watch") or []
        exchange = ccxt.binance()
        for sym in symbols:
            pretty = sym[:-4] + "/" + sym[-4:] if sym.endswith("USDT") else sym
            try:
                df_1d = fetch_ohlcv(exchange, pretty, "1d")
                text, _, status = analyze_one_timeframe(df_1d, "روزانه", pretty,
                    if status in ("buy", "sell") and last_alert_status.get(sym) != st
                        tg_send_message(f"
```

```
# =====
```

```
HELP_TEXT = (  
    "دستورات:\n"  
    "/analyze BTC/USDT - نمودار + تحلیل کامل\n"  
    "/watch SOLUSDT - اضافه کردن به رصد نهنگ\n"  
    "/unwatch SOLUSDT - حذف از رصد نهنگ\n"  
    "/watchlist - نمایش لیست رصد\n"  
    "/whales_on یا /whales_off - هشدار نهنگ کردن روشن/خاموش کردن هشدار نهنگ\n"  
    "/help - همین راهنما -"  
)
```

```
def handle_command(text, chat_id):  
    parts = text.strip().split()  
    if not parts:  
        return  
    cmd = parts[0].lower()  
  
    if cmd == "/start":  
        with state_lock:  
            state["chat_id"] = chat_id  
            save_state(state)  
            tg_send_message("سلام! ربات فعال شد. ✅\n\n" + HELP_TEXT, chat_id=chat_id)  
  
    elif cmd == "/help":  
        tg_send_message(HELP_TEXT, chat_id=chat_id)  
  
    elif cmd == "/analyze":  
        if len(parts) < 2:  
            tg_send_message("مثال: /analyze BTC/USDT", chat_id=chat_id)  
            return  
        symbol = parts[1].upper()  
        if "/" not in symbol:  
            symbol += "/USDT"  
        tg_send_message(f"در حال تحلیل {symbol} ... ⌚", chat_id=chat_id)  
        try:  
            run_full_analysis_and_send(symbol, chat_id)  
        except Exception as e:  
            tg_send_message(f"خطا: {e}", chat_id=chat_id)  
  
    elif cmd == "/watch":  
        if len(parts) < 2:  
            tg_send_message("مثال: /watch SOLUSDT", chat_id=chat_id)  
            return  
        sym = parts[1].upper().replace("/", "")  
        with state_lock:
```

```

        if sym not in state["watch"]:
            state["watch"].append(sym)
            save_state(state)
        tg_send_message(f"{sym} به رصد نهنگ اضافه شد.", chat_id=chat_id)
        if ws_thread_ref["current"]:
            ws_thread_ref["current"].close()

elif cmd == "/unwatch":
    if len(parts) < 2:
        tg_send_message("مثال: /unwatch SOLUSDT", chat_id=chat_id)
        return
    sym = parts[1].upper().replace("/", "")
    with state_lock:
        if sym in state["watch"]:
            state["watch"].remove(sym)
            save_state(state)
        tg_send_message(f"{sym} از رصد نهنگ حذف شد.", chat_id=chat_id)
        if ws_thread_ref["current"]:
            ws_thread_ref["current"].close()

elif cmd == "/watchlist":
    tg_send_message("نمادهای تحت رصد:\n" + "\n".join(state.get("watch", [])),

elif cmd == "/whales_on":
    with state_lock:
        state["whales_enabled"] = True
        save_state(state)
    tg_send_message("هشدار نهنگ فعال شد. ✅", chat_id=chat_id)

elif cmd == "/whales_off":
    with state_lock:
        state["whales_enabled"] = False
        save_state(state)
    tg_send_message("هشدار نهنگ غیرفعال شد.", chat_id=chat_id)

else:
    tg_send_message("/help بزن نشد. دستور شناخته نشد", chat_id=chat_id)

def telegram_polling_loop():
    last_update_id = 0
    print("... در حال گوش دادن به پیام‌ها [تلگرام]")
    while True:
        try:
            resp = requests.get(f"{TELEGRAM_API}/getUpdates", params={
                "offset": last_update_id + 1, "timeout": 30
            }, timeout=35)

```

```

        data = resp.json()
        for update in data.get("result", []):
            last_update_id = update["update_id"]
            msg = update.get("message", {})
            text = msg.get("text", "")
            chat_id = msg.get("chat", {}).get("id")
            if text and chat_id:
                handle_command(text, chat_id)
        except Exception as e:
            print(f"[تلگرام] خطا: {e}")
            time.sleep(5)

# =====
# اجرای برنامه
# =====

def main():
    if "اینجا توکن" in TELEGRAM_BOT_TOKEN:
        print("بذاری TELEGRAM_BOT_TOKEN اول باید توکن بات تلگرامت رو توی متغیر ⚠")
        sys.exit(1)

    print("... ربات در حال اجراست")
    print(f"نمادهای تحت رصد نهنگ: {state.get('watch')}")
    print("رو بزن تا فعال بشه /start توی تلگرام")

    threading.Thread(target=telegram_polling_loop, daemon=True).start()
    threading.Thread(target=whale_watcher_loop, daemon=True).start()
    threading.Thread(target=auto_analysis_loop, daemon=True).start()

    while True:
        time.sleep(60)

if __name__ == "__main__":
    main()

```